# Data-Structures & Algorithms

*with*

Manoj Prabhakaran

**Lecture 2**

**Asymptotics**

*Measuring the Trends*

# Course Logistics

- For the Lab today, find out group you are assigned to at https://bit.ly/A26-DSA-Mapping (needs IITB Google Suite login)

# Recall Our Experiment

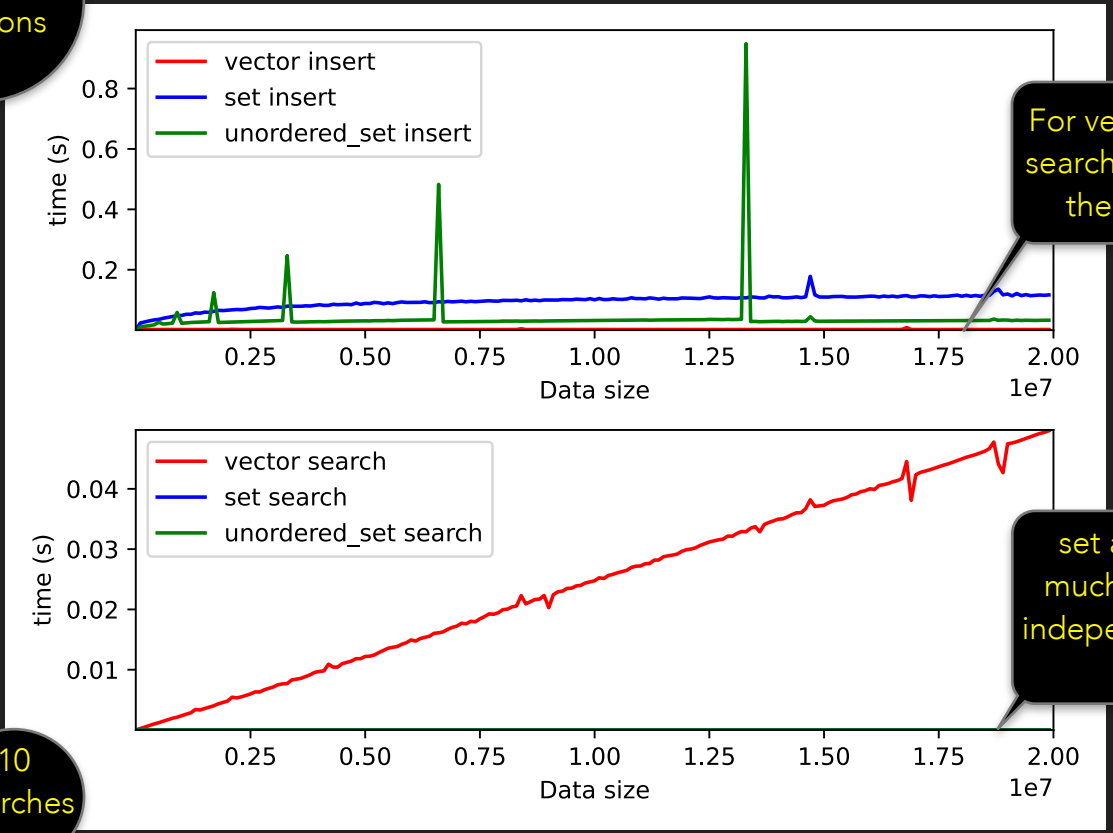
# Store and Retrieve


```
std::vector<int> v;
std::set<int> s;
std::unordered_set<int> u;
...
// in a loop
  v.push_back(item);
  s.insert(item);
  u.insert(item);
...
std::find(v.begin(), v.end(), query); //iterate through the range
s.find(query);
u.find(query);
```

# Store and Retrieve

Recall

100000 insertions




For vector, insert is very fast, but search gets slower and slower as the number of items grows

set and unordered_set have much faster search, seemingly independent of how big the data set is

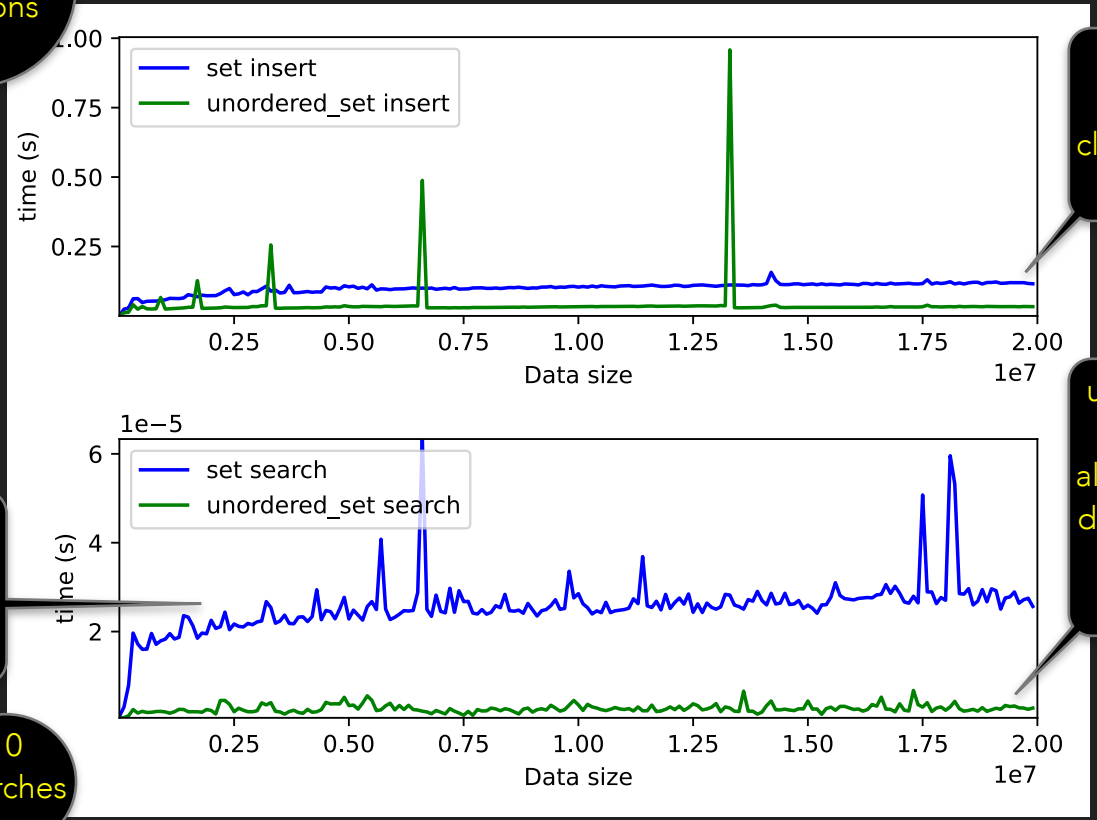
10 searches

# Store and Retrieve

Recall

100000 insertions

A closer look at set vs. unordered_set




set insert is also slowly climbing with the data size

unordered_set insert and search both take time almost independent of the data-size (except for insert times spiking at various data sizes)

set search is slowly climbing with the data size

10 searches

# Measuring Behaviour

- What is under the hood of std::set and std::unordered_set, and why do they behave so differently from each other?
- But first: we need some vocabulary to discuss their behaviour
  - Asymptotic Worst-Case upper bounds
- Let us unpack that

# Asymptotic Behaviour

# Asymptotic Behaviour

- Imagine a quantity, that starts growing starting from 1 item on day 0, in one of three ways
  - Each day, increases by a million: 1, 1000001, 2000001, ...
  - On day n, increases by 10000n: 1, 10001, 30001, 60001, ...
  - Each day, doubles: 1, 2, 4, 8, ...

$A(n) = 10^6 n + 1$

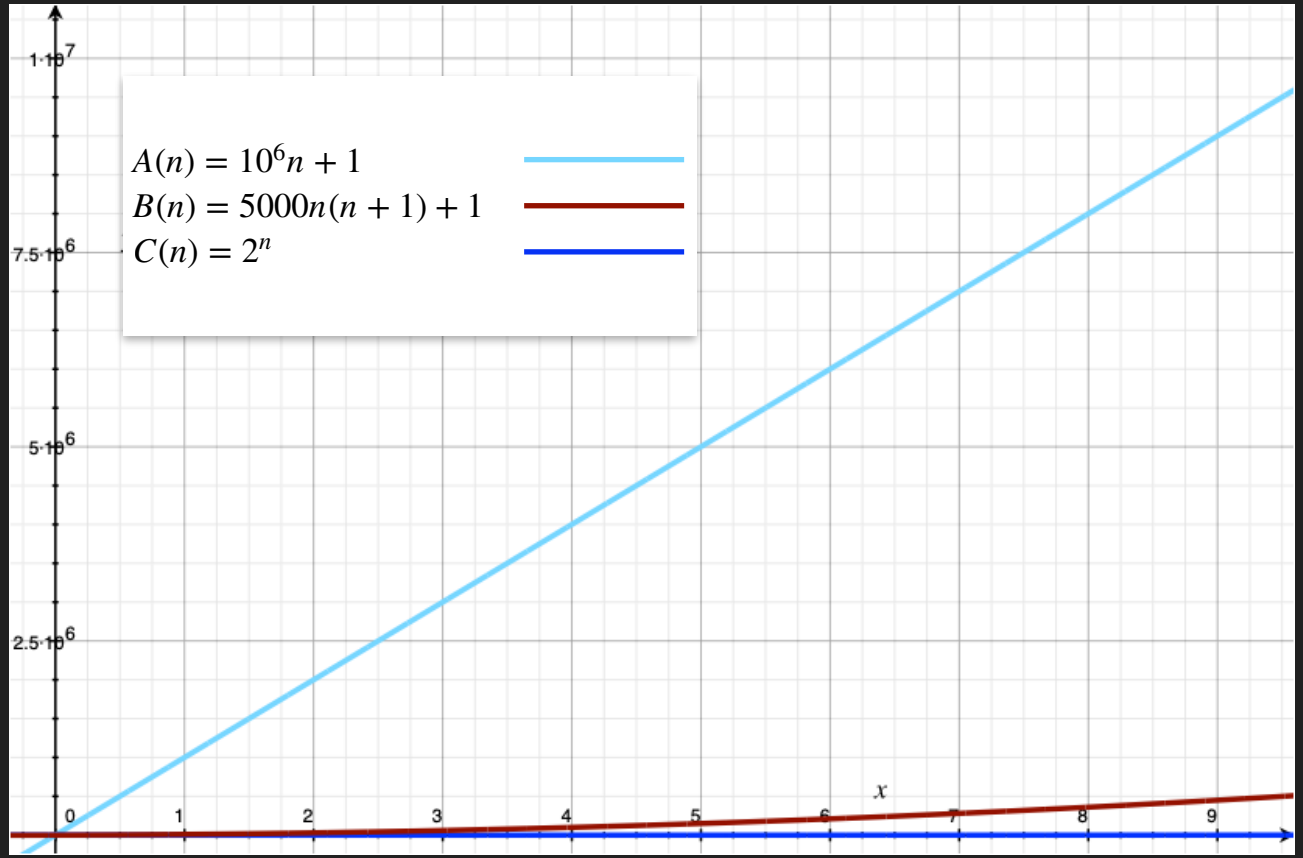
$B(n) = 10000n(n+1)/2 + 1$

$C(n) = 2^n$

- After a week, which rule will result in the largest amount? After a month?

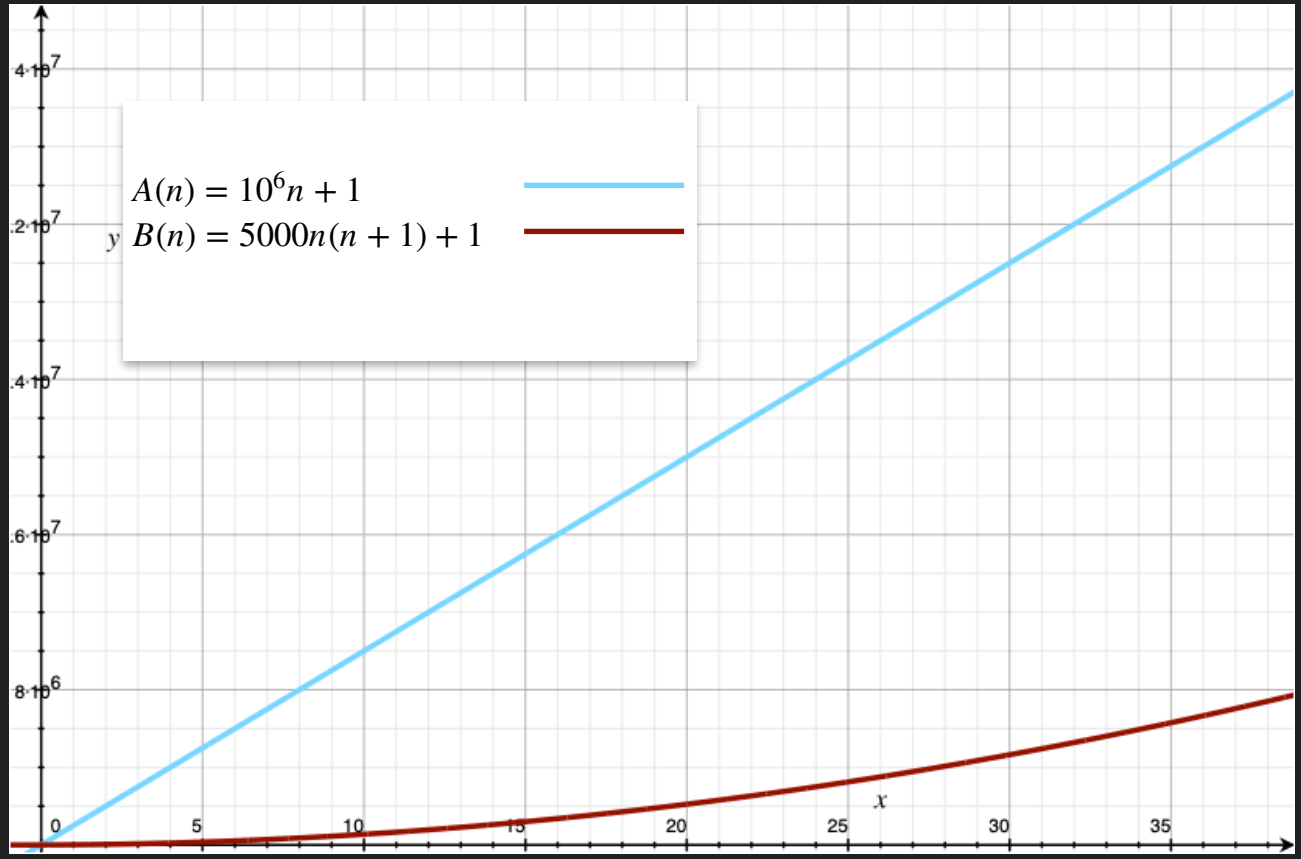
# Asymptotic Behaviour

The first couple of weeks



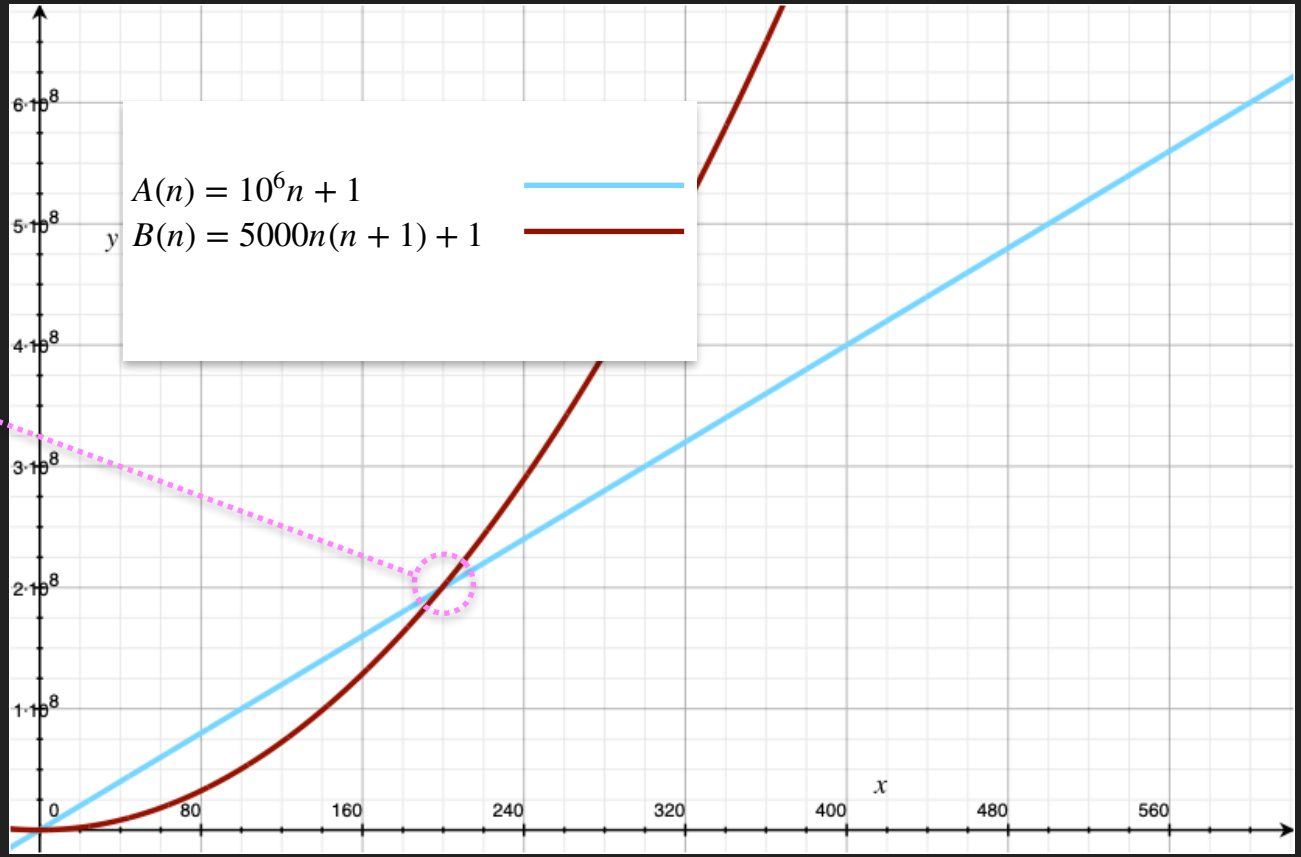
# Asymptotic Behaviour

A vs. B:
First month



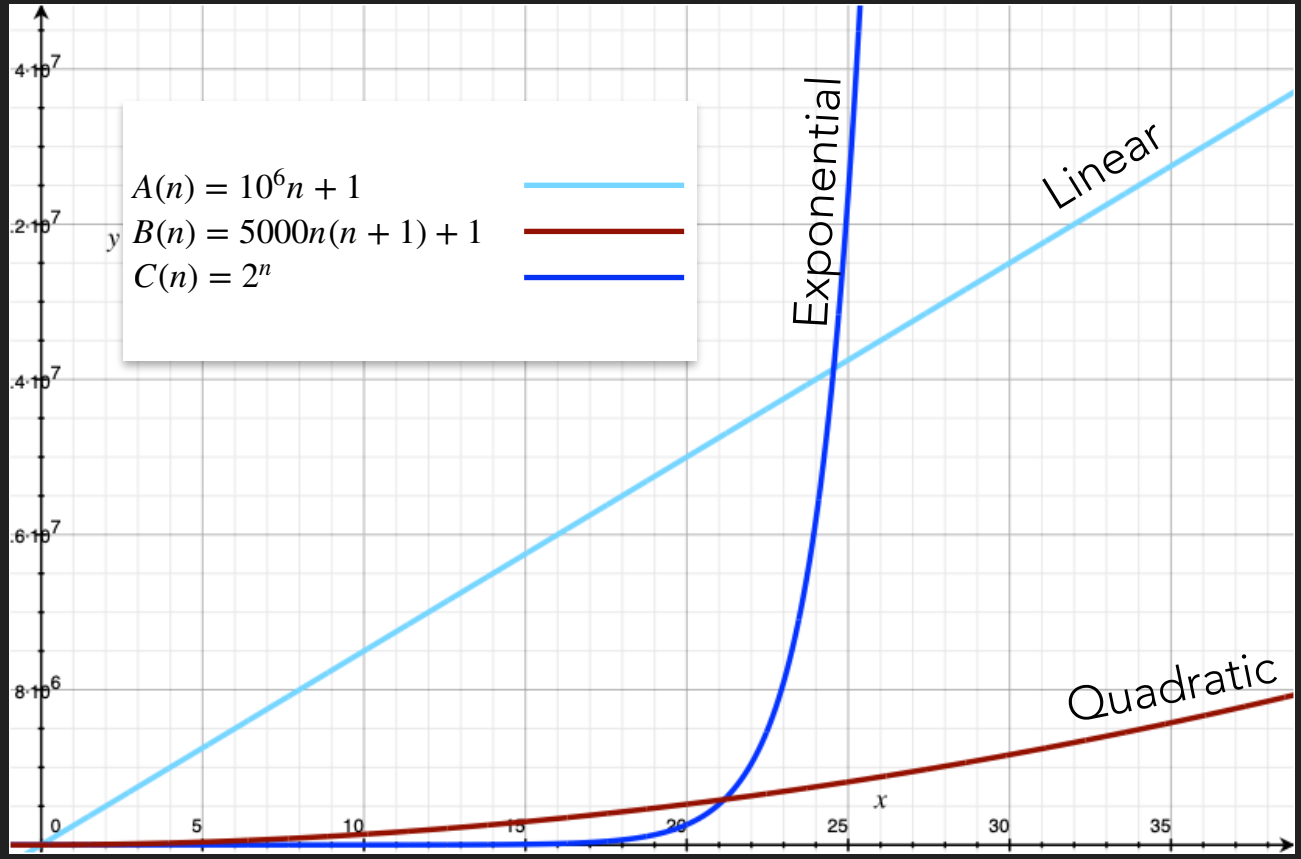
# Asymptotic Behaviour

A vs. B:
200 days later



# Asymptotic Behaviour

Back to the first month




Eventually,

Exponential
>> Quadratic
>> Linear

# Asymptotic Behaviour

- *We say* f(n) has an asymptotically linear upper bound, if there is a scaling factor $c > 0$ and a threshold $k > 0$, such that for all $n > k$, $f(n) \leq c.n$
  - The "Big-Oh" notation: $f(n) = O(n)$
- And a quadratic upper bound, or $f(n) = O(n^2)$, if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.n^2$
- More generally, $f(n) = O(g(n))$ if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.g(n)$
- E.g., Suppose $f(n) = 5n^2 + 2n + 1$.
  - Then $f(n) = O(n^2)$. $5n^2 + 2n + 1 \leq 6n^2$ once $2n+1 \leq n^2$, or $n \geq 3$
  - But f(n) is not O(n).

If $5n^2 + 2n + 1 \leq cn$ for some c, then $n^2 \leq cn$. But this holds only for $n \leq c$, and not all $n \geq$ any k.

# Asymptotic Behaviour

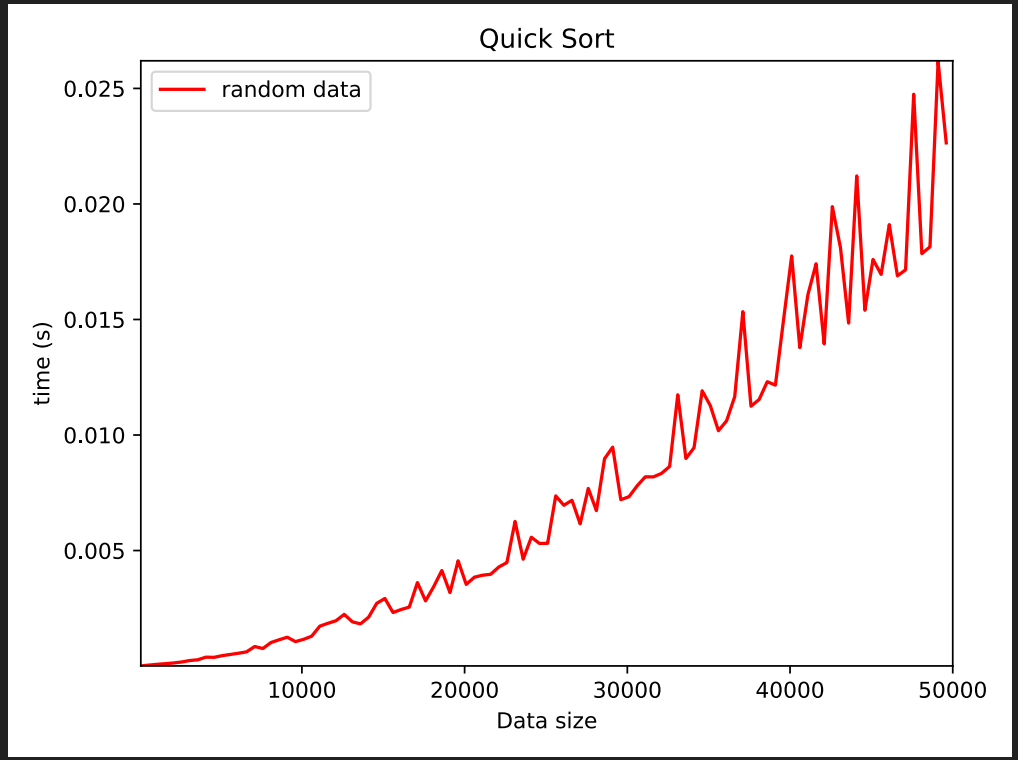
- We are typically interested in the question of scaling (to very large inputs)
  - E.g., If processing a query involving million data points takes 1s, how long will it take to process one involving 10 million data points?
    - Will it take  about the same, 10x more, or 100x more, or something else?
    - Depends on how the processing algorithm scales.
    - If processing time (as a function of input data size n) is O(n), then roughly 10 seconds. If it is $O(n^2)$, roughly 100 seconds.

# Asymptotic Behaviour

- $O(\cdot)$ is an upper bound
  - E.g., $f(n) := 5n^2 + 2n + 1 = O(n^2)$. But it is also $O(n^3)$, $O(n^4)$ etc.
  - If $f(n) = O(g(n))$ and $g(n)=O(f(n))$, we write $f(n) = \Theta(g(n))$
    - Equivalently, $f(n) = \Theta(g(n))$ iff there exist positive constants $\alpha$, $\beta$, $k$ s.t. for all $n \geq k$, $\alpha.g(n) \leq f(n) \leq \beta.g(n)$
- But what would f be?
  - Running time of an algorithm (in some abstract machine). But on which input? Worst-Case!

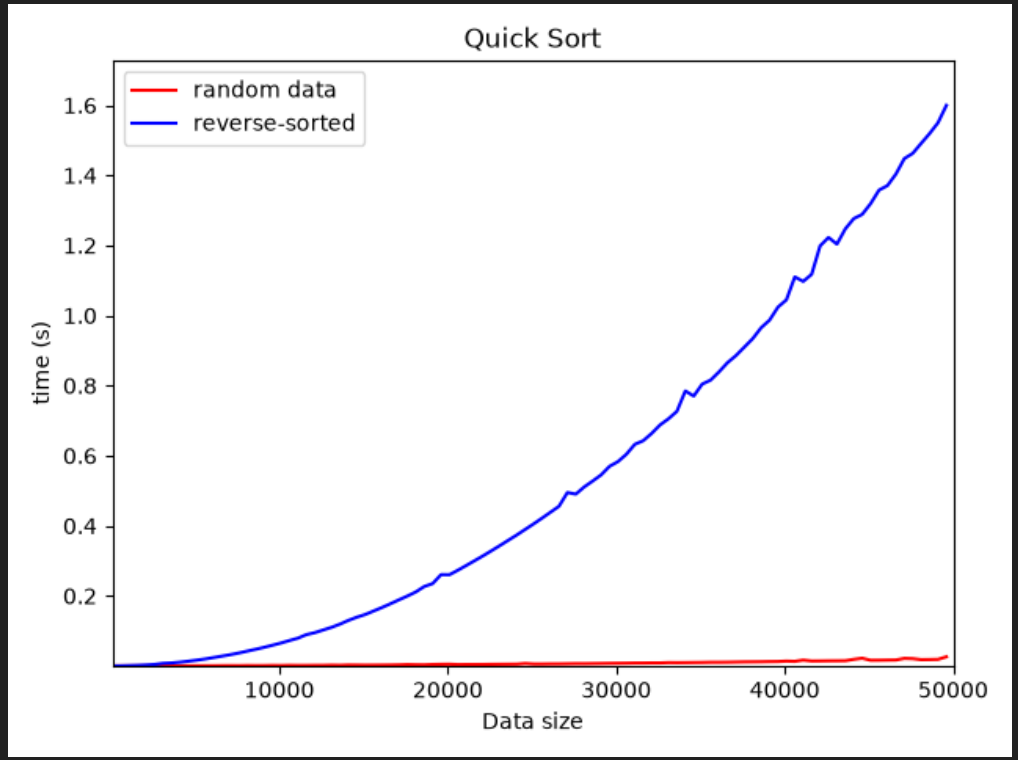
# An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)



# An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)
- On reverse sorted data, takes *much* longer
  - $\Theta(n^2)$



# Asymptotic Worst-Case Time

- Asymptotic:  What is the trend as data grows
- Worst-Case: Algorithms can perform wildly differently on different kinds of inputs. We should ideally be prepared for the worst-case.
  - Calculate O(.) upper bounds or Θ(.) bounds *analytically.*
- Some examples we will encounter:  O(1), O(log n), O(n), O(n log n), $O(n^2)$
  - std::set has Θ(log n) insertion and search  (where n is the current set size)
  - std::unordered_set has Θ(1) search; insert can occasionally be Θ(n).
  - std::vector has Θ(1) insert and Θ(n) search.